



Fine-Tuning Large Language Models

Tutorial: Fine-tuning a Language Model and Deploying with Hugging Face Spaces - Sentiment Analysis IMDB Reviews

What is Fine-Tuning?

Fine-tuning means taking a large language model (LLM) that was already trained on a massive text corpus, and continuing to train it on your own smaller, task-specific dataset.

The Transfer Learning Analogy

Think of a pre-trained LLM as a university graduate with a deep general education — they already understand language, grammar, context, and world knowledge.

Fine-tuning is like sending that graduate on a short, focused training course (e.g., "classify movie sentiment") so they become specialised for your specific task. This is far more efficient than educating someone from scratch.

Why Not Train from Scratch?

Approach	Data Required	Compute Required	Time
Train from scratch	Billions of tokens	Thousands of GPUs	Weeks
Fine-tune a pre-trained model	Hundreds–thousands of examples	1 GPU (or even CPU)	Minutes–hours

What Happens Technically?

1. Pre-trained model weights (learned from billions of tokens) are loaded as the starting point
2. A **task-specific head** is added on top — e.g., a linear layer with 2 outputs for positive/negative
3. The model trains on your labelled data — all weights are adjusted via backpropagation
4. The result retains broad language understanding and is now specialised for your task

This Tutorial: IMDb Sentiment Classification

- **Input:** A movie review (free text)
- **Output:** Positive or Negative label
- **Model:** DistilBERT — a compact version of BERT (40 % smaller, 60 % faster, ~97 % of BERT's accuracy)
- **Dataset:** IMDb movie reviews — we use 2,000 of the 25,000 available training examples for speed

Step 1: Install Required Libraries

We install the Python libraries needed for this entire workflow:

Library	Purpose
<code>transformers</code>	Hugging Face library — provides pre-trained models, tokenizers, and the <code>Trainer</code> API
<code>datasets</code>	Easily download and process standard benchmark datasets
<code>huggingface_hub</code>	Authenticate and upload models/tokenizers to the Hugging Face Hub
<code>gradio</code>	Build an interactive web demo for your model with just a few lines of Python
<code>scikit-learn</code>	Compute evaluation metrics (accuracy, F1, precision, recall)

```
In [ ]: # transformers - pre-trained models, tokenizers, and the Trainer API
# datasets - easy access to benchmark datasets hosted on the HF Hub
# huggingface_hub - login and push models to huggingface.co
# gradio - build a quick web demo in a few lines of Python
# scikit-learn - evaluation metrics: accuracy, F1, precision, recall
! pip install transformers datasets huggingface_hub gradio scikit-learn
```

Step 2: Load and Prepare the Dataset

We use the **IMDb dataset** — a standard benchmark for binary sentiment classification:

- 50,000 movie reviews (25,000 train / 25,000 test), each labeled **positive (1)** or **negative (0)**
- Example review: *"This movie was absolutely fantastic!"* → label: **Positive (1)**

To keep training fast in this tutorial, we only load **2,000 of the 25,000 training reviews**. We then split those 2,000 samples into:

- **80% = 1,600 samples** → training set — the model learns from these
- **20% = 400 samples** → validation set — used to measure performance, never seen during learning

```
In [ ]: from datasets import load_dataset
```

```
# Load the first 2,000 of the 25,000 available training reviews.
# The slice syntax "train[:2000]" works like Python list slicing.
# Using a small subset keeps training time manageable on free Colab GPUs.
dataset = load_dataset("imdb", split="train[:2000]")

# Split into 80 % training (1,600 samples) and 20 % validation (400 samples).
# The validation split is used to measure generalisation – the model never
# learns from these examples directly, only from the training split.
dataset = dataset.train_test_split(test_size=0.2)

print(dataset) # shows the split sizes
print("\nExample entry:")
print(dataset["train"][0]) # shows what a single labelled review looks like
```

Step 3: Load the Tokenizer and Model

This step loads two objects: a **tokenizer** and a **model**.

AutoTokenizer

Raw text cannot be fed directly into a neural network — it must first be converted to numbers.

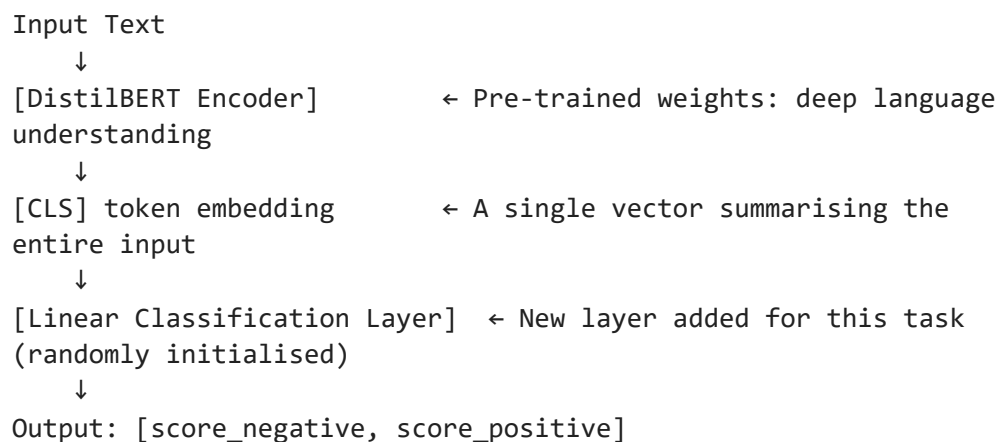
The tokenizer does this in two steps:

1. **Splits text into tokens** — sub-word pieces from a fixed vocabulary of ~30,000 tokens
Example: "playing" → ["play", "##ing"]
2. **Converts each token to an integer ID** from the vocabulary
Example: ["play", "##ing"] → [2377, 6370]

`AutoTokenizer.from_pretrained(checkpoint)` automatically downloads the correct tokenizer for the model you specify — no need to look up which class to use manually.

AutoModelForSequenceClassification

This loads a pre-trained encoder model with a **classification head** attached on top:



The `num_labels=2` argument tells it to build a 2-class output layer (positive / negative). The pre-trained encoder keeps its learned language knowledge; only the new classification layer starts from scratch.

Why DistilBERT?

DistilBERT is a *distilled* (compressed) version of BERT, created by training a smaller model to mimic BERT's behaviour (a technique called **knowledge distillation**):

Model	Parameters	Inference Speed	GLUE Score
BERT-base	110M	1×	100%
DistilBERT	66M	1.6× faster	~97%

It is the ideal teaching model: fast enough to fine-tune in minutes on a free GPU, accurate enough to be meaningful.

```
In [ ]: from transformers import AutoTokenizer, AutoModelForSequenceClassification

# The checkpoint name identifies which pre-trained model to download.
# "distilbert-base-uncased" → DistilBERT, trained on Lowercased English text.
checkpoint = "distilbert-base-uncased"

# Downloads the vocabulary and tokenisation rules that match this model.
# The tokenizer must always match the model – mismatching them would produce
# incorrect token IDs and garbage predictions.
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

# Downloads the pre-trained DistilBERT encoder and appends a fresh
# linear classification layer on top.
# num_labels=2 → two output neurons: one for "negative", one for "positive"
# The encoder weights come pre-trained; only the new classification layer
# is randomly initialised and will be learned from our IMDb data.
model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)
```

Step 4: Tokenize the Dataset

Before the model can process text, each review must be converted into numerical tensors. The tokenizer produces two arrays for every input:

Output	Description	Example (simplified)
<code>input_ids</code>	Integer ID for each token in the vocabulary	<code>[101, 2023, 3185, 102, 0, 0, ...]</code>
<code>attention_mask</code>	1 for real tokens, 0 for padding tokens	<code>[1, 1, 1, 1, 0, 0, ...]</code>

Key parameters used here:

- `padding="max_length"` — Pads all sequences to the same length (512 tokens). Short reviews get zeros appended so every batch has a uniform shape.
- `truncation=True` — Reviews longer than 512 tokens are cut off at exactly 512.
- `batched=True` in `.map()` — Processes examples in chunks rather than one-by-one, which is significantly faster.

`set_format("torch", ...)` converts the dataset columns into PyTorch tensors, which is the format the `Trainer` expects.

```
In [ ]: def tokenize_function(example):
        """Convert a batch of raw review strings into token IDs and attention masks."""
        return tokenizer(
            example["text"],
            padding="max_length", # pad shorter reviews with zeros to reach 512 tokens
            truncation=True,      # cut reviews longer than 512 tokens
        )

# Apply tokenize_function to every example in both splits.
# batched=True sends a chunk of examples to the function at once – much faster
# than calling the tokenizer one review at a time.
tokenized_dataset = dataset.map(tokenize_function, batched=True)

# Tell the dataset to return PyTorch tensors for these three columns.
# The Trainer expects tensors, not Python Lists.
#   input_ids      – token IDs (what the model reads)
#   attention_mask – which positions are real tokens vs. padding
#   label          – ground truth: 0 (negative) or 1 (positive)
tokenized_dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"])
```

Step 5: Define Training Arguments and Trainer

`TrainingArguments` controls *how* training runs — all the hyperparameters and logistics in one place:

Parameter	Value	What it controls
<code>output_dir</code>	<code>"./results"</code>	Folder where model checkpoints are saved
<code>eval_strategy</code>	<code>"epoch"</code>	Evaluate on the validation set at the end of every epoch
<code>num_train_epochs</code>	<code>3</code>	Number of full passes through the training data
<code>per_device_train_batch_size</code>	<code>8</code>	How many examples to process together in one forward pass during training
<code>per_device_eval_batch_size</code>	<code>8</code>	Same, but during evaluation
<code>save_total_limit</code>	<code>1</code>	Keep only the most recent checkpoint (saves disk space)

Trainer wraps the entire training loop so you do not have to write it manually:

1. Forward pass (input → logits)
2. Loss computation — **cross-entropy loss** between predicted and true labels
3. Backward pass — compute gradients with respect to the loss
4. Weight update — adjust all model parameters via the Adam optimiser

compute_metrics defines what we measure after each epoch:

- **Accuracy** — fraction of predictions that are correct
- **F1 score** — harmonic mean of precision and recall (better than accuracy for potentially imbalanced data)
- **Precision** — of everything predicted positive, how many actually were?
- **Recall** — of all true positives, how many did we correctly catch?

```
In [ ]: from transformers import TrainingArguments, Trainer
import numpy as np
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

training_args = TrainingArguments(
    output_dir="./results",          # save model checkpoints here
    eval_strategy="epoch",          # evaluate on the validation set after each epoch
    num_train_epochs=3,             # 3 full passes over the 1,600 training examples
    per_device_train_batch_size=8,  # process 8 reviews at a time during training
    per_device_eval_batch_size=8,   # process 8 reviews at a time during evaluation
    save_total_limit=1,             # keep only the most recent checkpoint
    logging_dir="./logs",           # TensorBoard-compatible training logs
)

def compute_metrics(eval_pred):
    """Compute classification metrics from model outputs at the end of each epoch."""
    logits, labels = eval_pred
    # Logits: raw model scores, shape (n_samples, 2) – one score per class
    # argmax along axis=1 selects the class with the higher score
    predictions = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="binary"
    )
    acc = accuracy_score(labels, predictions)
    return {"accuracy": acc, "f1": f1, "precision": precision, "recall": recall}

trainer = Trainer(
    model=model,                    # the DistilBERT model with classification head
    args=training_args,             # hyperparameters defined above
    train_dataset=tokenized_dataset["train"], # 1,600 examples the model learns from
    eval_dataset=tokenized_dataset["test"],   # 400 examples used only for evaluation
    compute_metrics=compute_metrics,         # called automatically after each epoch
)
```

Step 6: Train the Model

This single call starts the full training loop. The `Trainer` will:

1. Iterate through the 1,600 training reviews in mini-batches of 8
2. Compute the **cross-entropy loss** between model predictions and true labels
3. Backpropagate gradients and update all model weights
4. At the end of each epoch, evaluate on the 400 validation reviews and print the metrics

Expected runtime: ~5–15 minutes on a free Colab T4 GPU. On CPU it may take 30–60 minutes.

After 3 epochs you should see accuracy above **85%** — even with only 2,000 examples. This demonstrates the power of starting from a pre-trained model: it already understands language; we only need to teach it the classification task.

```
In [ ]: trainer.train()
```

Step 7: Log in to Hugging Face Hub

The **Hugging Face Hub** is a platform (like GitHub, but for ML models) where you can:

- Share your trained model publicly or privately
- Load it from anywhere with `pipeline("sentiment-analysis", model="your-username/model-name")`
- Build web demos that anyone can try

This step authenticates you using a **personal access token**.

You will need a free Hugging Face account and a token with **write** permission — see the link below.

```
In [ ]: from huggingface_hub import notebook_login

notebook_login()
```

After running this cell, you'll be prompted to enter your Hugging Face access token. You can create one here: <https://huggingface.co/settings/tokens>

Step 8: Push Model and Tokenizer to Hugging Face Hub

This uploads the fine-tuned model weights and the tokenizer vocabulary to the Hub.

After pushing, anyone (or your Gradio app) can load the model with:

```
pipeline("sentiment-analysis", model="your-username/distilbert-sentiment-imdb-small")
```

Replace `"your-username"` with your actual Hugging Face username before running this cell.

```
In [ ]: model_name = "your-username/distilbert-sentiment-imdb-small"

model.push_to_hub(model_name)
tokenizer.push_to_hub(model_name)
```

Step 9: (Optional) Test with Gradio Locally in Colab

Before deploying publicly, you can launch a temporary Gradio interface directly inside Colab. This lets you type a review and see the model's prediction immediately — useful for checking that the model actually works before uploading it anywhere.

The `interface.launch()` call opens a shareable URL that stays active as long as the Colab session runs.

```
In [ ]: import gradio as gr
from transformers import pipeline

classifier = pipeline("sentiment-analysis", model=model_name)

def predict_sentiment(text):
    result = classifier(text)[0]
    return f"Label: {result['label']}, Confidence: {round(result['score'], 3)}"

interface = gr.Interface(
    fn=predict_sentiment,
    inputs="text",
    outputs="text",
    title="Sentiment Analysis",
    description="Enter a movie review to classify as POSITIVE or NEGATIVE."
)

interface.launch()
```

Step 10: Deploy as a Hugging Face Space

A **Hugging Face Space** is a free hosted web app that anyone can visit in their browser.

Setup steps:

1. Go to <https://huggingface.co/spaces> and click **Create New Space**
2. Choose **Gradio** as the SDK and give it a name (e.g. `sentiment-analyzer-student`)
3. Create two files in the Space repository:

File 1 — `app.py` (the application code — copy the cell below into this file):

```
# app.py – upload this file to your Hugging Face Space
import gradio as gr
from transformers import pipeline

model_name = "your-username/distilbert-sentiment-imdb-small"
```



```

classifier = pipeline("sentiment-analysis", model=model_name)

def predict_sentiment(text):
    result = classifier(text)[0]
    return f"Label: {result['label']}, Confidence: {round(result['score'],
3)}"

interface = gr.Interface(
    fn=predict_sentiment,
    inputs="text",
    outputs="text",
    title="Sentiment Analysis",
    description="Enter a movie review to classify as POSITIVE or
NEGATIVE."
)

interface.launch()

```

File 2 — requirements.txt (tells Hugging Face which packages to install):

```

transformers
torch
gradio

```

After uploading both files, Hugging Face automatically builds and deploys your Space.

The public URL will be: <https://huggingface.co/spaces/your-username/sentiment-analyzer-student>

Conclusion

This complete workflow demonstrates how to:

- Fine-tune a transformer model on a small task-specific dataset using the Hugging Face `Trainer` API
- Save and share the fine-tuned model via the Hugging Face Hub
- Deploy the model as an interactive web app with Gradio and Hugging Face Spaces

The same pattern — load a pre-trained checkpoint, tokenize your data, define `TrainingArguments`, train — applies to almost any text classification task. Swap the dataset, adjust `num_labels`, and you have a new classifier.

Going Further: Parameter-Efficient Fine-Tuning (PEFT)

The approach in this tutorial updates **all** of the model's weights during training — known as **full fine-tuning**. For larger modern models (Mistral 7B, LLaMA 3, Phi-3), this becomes

impractical: it requires enormous GPU memory and long training times.

Parameter-Efficient Fine-Tuning (PEFT) methods solve this by freezing most of the model and only training a very small number of new parameters:

Method	How it works	Trainable parameters
LoRA	Inserts small low-rank matrices alongside the existing weight matrices	~0.1–1 % of total
QLoRA	LoRA + 4-bit quantisation — fits a 7B model on a single consumer GPU	~0.1–1 % of total
Prefix Tuning	Prepends a set of learnable tokens to every layer	Very few

Example: Fine-Tuning with LoRA (using the `peft` library)

```
from peft import get_peft_model, LoraConfig, TaskType

lora_config = LoraConfig(
    task_type=TaskType.SEQ_CLS, # we are doing sequence classification
    r=8, # rank of the low-rank matrices (higher =
more capacity)
    lora_alpha=32, # scaling factor for the LoRA updates
    lora_dropout=0.1, # dropout applied inside the LoRA layers
)

# Wraps the existing model — only the LoRA layers will be trained
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable params: 294,912 || all params: 66,955,010 || trainable%: 0.44
You then pass this wrapped model to Trainer exactly as before — nothing else changes.
```

LoRA and QLoRA are now the standard approach for fine-tuning modern LLMs on consumer hardware.

Example 2 — Fine-Tuning BERT on Amazon Reviews

This example uses **full BERT** (instead of DistilBERT) on the **Amazon Polarity** dataset:

- Over 3.6 million Amazon product reviews — we use 2,000 for speed
- Each review has a `title`, a `content` field, and a binary `label` (0 = negative, 1 = positive)
- We combine the `title` and `content` into a single input string

Key difference from Example 1: We use `BertTokenizer` and

`BertForSequenceClassification` directly

instead of the `Auto` classes. Both approaches work identically — the `Auto` classes simply select

the correct class automatically based on the checkpoint name. Specifying the class directly is useful when you want to be explicit about exactly which architecture you are using.

```
In [ ]: from transformers import BertTokenizer, BertForSequenceClassification, Trainer, Tra
from datasets import load_dataset
from sklearn.metrics import accuracy_score
import numpy as np

# Load 2,000 samples from the Amazon Polarity dataset directly from Hugging Face.
# This avoids any dependency on a local CSV file.
# Columns: "title" (short heading), "content" (full review text), "label" (0=neg, 1
raw_dataset = load_dataset("amazon_polarity", split="train[:2000]")
raw_dataset = raw_dataset.train_test_split(test_size=0.2)

# BertTokenizer is the explicit class for BERT's tokenizer.
# Equivalent to AutoTokenizer.from_pretrained("bert-base-uncased").
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

def tokenize_function(example):
    """Combine title + content into one string, then tokenize."""
    # Concatenating title and content gives the model more context per review.
    combined = [t + " " + c for t, c in zip(example["title"], example["content"])]
    # max_length=256 (half of BERT's max 512) – keeps memory usage lower
    # while still capturing most of the review text.
    return tokenizer(combined, padding="max_length", truncation=True, max_length=256)

tokenized_data = raw_dataset.map(tokenize_function, batched=True)
tokenized_data.set_format(type="torch", columns=["input_ids", "attention_mask", "la

# BertForSequenceClassification is the explicit class equivalent of
# AutoModelForSequenceClassification.from_pretrained("bert-base-uncased", num_label
# Full BERT has 110M parameters – larger and more accurate than DistilBERT, but slo
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labe

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1) # convert raw scores to class predictions
    return {"accuracy": accuracy_score(labels, preds)}

training_args = TrainingArguments(
    output_dir="./results-bert-amazon",
    eval_strategy="epoch",
    save_strategy="epoch",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    logging_dir="./logs",
    logging_steps=10,
    # print a log line every 10 training steps
)
```

```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_data["train"],
    eval_dataset=tokenized_data["test"],
    compute_metrics=compute_metrics,
)

trainer.train()

# Save the fine-tuned model weights and tokenizer to a local folder.
# You can later reload both with:
# model = BertForSequenceClassification.from_pretrained("fine_tuned_bert_amazon_r
# tokenizer = BertTokenizer.from_pretrained("fine_tuned_bert_amazon_reviews")
trainer.save_model("fine_tuned_bert_amazon_reviews")
tokenizer.save_pretrained("fine_tuned_bert_amazon_reviews")

```

Reference: More Fine-Tuning Examples

The code blocks below are **reference examples** — they are not meant to be run step-by-step in this notebook, but show how the same Hugging Face fine-tuning pattern extends to other NLP tasks.

#	Task	Type	Dataset	Model
1	News topic classification	Multi-class (4 labels)	AG News	DistilBERT
2	Text generation / story completion	Language modelling	WikiText-2	GPT-2
3	Named Entity Recognition (NER)	Token-level classification	CoNLL-2003	BERT-base-cased

1. Text Classification: News Topic Classification

Task: Classify news articles into topics (e.g., business, sports, politics)

- **Dataset:** AG News (4-class classification)
- **Model:** `distilbert-base-uncased`
- **Why it's good:** Multiclass instead of binary; introduces students to topic classification.

```

from datasets import load_dataset
from transformers import AutoTokenizer,
AutoModelForSequenceClassification, TrainingArguments, Trainer
import numpy as np
from sklearn.metrics import accuracy_score,
precision_recall_fscore_support

```

```

# Load data
dataset = load_dataset("ag_news")
dataset = dataset["train"].train_test_split(test_size=0.2)

```

```

# Tokenizer and model
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained("distilbert-
base-uncased", num_labels=4)

def tokenize_function(example):
    return tokenizer(example["text"], padding="max_length",
truncation=True)

tokenized_data = dataset.map(tokenize_function, batched=True)
tokenized_data.set_format("torch", columns=["input_ids", "attention_mask",
"label"])

# Training arguments
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    per_device_train_batch_size=8,
    num_train_epochs=3,
)

# Evaluation
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    acc = accuracy_score(labels, preds)
    return {"accuracy": acc}

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_data["train"],
    eval_dataset=tokenized_data["test"],
    compute_metrics=compute_metrics,
)

trainer.train()

```

Then follow the same login + push + deploy steps.

2. Text Generation: Simple Story Completion (using GPT-2)

Task: Given a prompt, generate the next few sentences of a story.

- **Model:** gpt2
- **Dataset:** A small set of fairy tales or a pre-tokenized open dataset like wikitext

GPT-based fine-tuning takes longer and needs GPU memory, so keep the dataset very small.

```

from datasets import load_dataset
from transformers import GPT2Tokenizer, GPT2LMHeadModel, Trainer,
TrainingArguments

# Load small dataset
dataset = load_dataset("wikitext", "wikitext-2-raw-v1",
split="train[:1%]")

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token # GPT2 has no padding token

def tokenize_function(examples):
    return tokenizer(examples["text"], return_special_tokens_mask=True,
truncation=True, padding="max_length", max_length=128)

tokenized_dataset = dataset.map(tokenize_function, batched=True)
tokenized_dataset.set_format("torch", columns=["input_ids"])

# Load model
model = GPT2LMHeadModel.from_pretrained("gpt2")

training_args = TrainingArguments(
    output_dir="./gpt2-finetuned",
    overwrite_output_dir=True,
    per_device_train_batch_size=2,
    num_train_epochs=1,
    save_total_limit=1,
    prediction_loss_only=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset,
)

trainer.train()

```

You can then build a Gradio app with a text input (prompt) and text output (generated continuation).

3. Named Entity Recognition (NER)

Task: Identify entities like person names, locations, etc.

- **Dataset:** conll12003
- **Model:** bert-base-cased

NER gives students exposure to **token-level** classification.

```

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForTokenClassification,

```

```

TrainingArguments, Trainer
from transformers import DataCollatorForTokenClassification
import numpy as np

dataset = load_dataset("conll12003")
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")

# Tokenize
def tokenize_and_align_labels(examples):
    tokenized_inputs = tokenizer(examples["tokens"], truncation=True,
is_split_into_words=True, padding="max_length")
    labels = []
    for i, label in enumerate(examples["ner_tags"]):
        word_ids = tokenized_inputs.word_ids(batch_index=i)
        previous_word_idx = None
        label_ids = []
        for word_idx in word_ids:
            if word_idx is None:
                label_ids.append(-100)
            elif word_idx != previous_word_idx:
                label_ids.append(label[word_idx])
            else:
                label_ids.append(label[word_idx])
            previous_word_idx = word_idx
        labels.append(label_ids)
    tokenized_inputs["labels"] = labels
    return tokenized_inputs

tokenized_data = dataset.map(tokenize_and_align_labels, batched=True)
tokenized_data.set_format("torch")

model = AutoModelForTokenClassification.from_pretrained("bert-base-cased",
num_labels=9)

training_args = TrainingArguments(
    output_dir="./ner-model",
    eval_strategy="epoch",
    per_device_train_batch_size=8,
    num_train_epochs=2,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_data["train"].select(range(1000)),
    eval_dataset=tokenized_data["validation"].select(range(200)),
    data_collator=DataCollatorForTokenClassification(tokenizer),
)

trainer.train()

```